RAG Chatbot with MCP

Code guide & architecture walkthrough for beginners

This document explains a Retrieval-Augmented Generation (RAG) chatbot that answers questions about the Model Context Protocol (MCP) specification. It covers the complete source code, the system architecture, and a step-by-step explanation written for developers who are new to LLMs and vector databases.

Section	Topic
1	Architecture diagram
2	Complete source code
3	Code walkthrough for beginners
4	How RecursiveCharacterTextSplitter works

1. Architecture Diagram

The system operates in two distinct phases. During startup it indexes the MCP PDF into a vector database. At query time it retrieves relevant passages using both semantic similarity and keyword matching, then asks the LLM to answer.

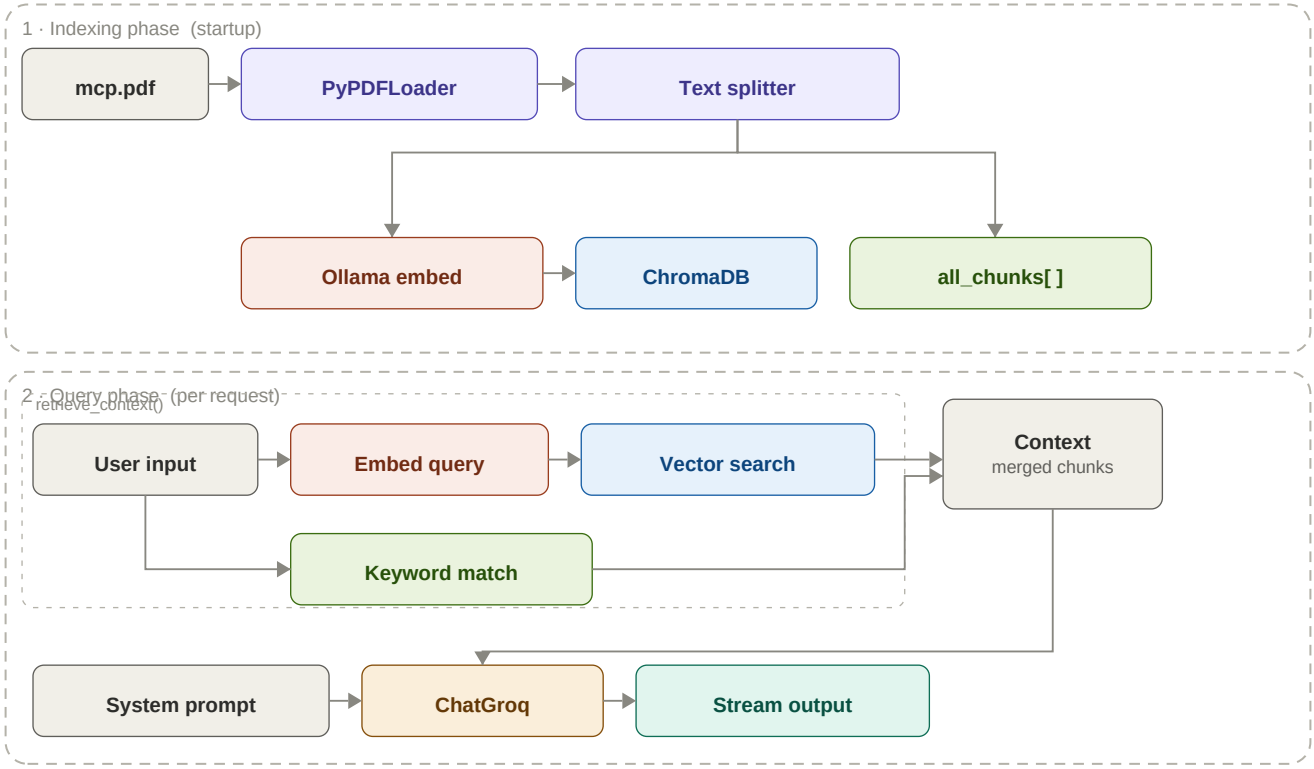

Figure 1 — Two-phase RAG architecture. Purple = LangChain loaders, coral = Ollama embedding, blue = vector store, green = keyword store, amber = Groq LLM, teal = streamed output.

Component	Library / Service	Role
PyPDFLoader	LangChain Community	Reads the PDF into Document objects
Text splitter	LangChain Text Splitters	Splits docs into 1000-char chunks with 200-char overlap
Ollama embed	Ollama (local)	Turns text into embedding vectors (e.g. nomic-embed-text)
ChromaDB	Chroma (in-memory)	Stores and searches embedding vectors
all_chunks[]	Python list	Plain-text copy of every chunk for keyword search
ChatGroq	Groq API	Calls the LLM (e.g. llama3) and streams the response

2. Complete Source Code

The complete file is shown below. Section 3 walks through each part.

```
from dotenv import load_dotenv
from os import getenv
import os
import re
import ollama
import chromadb
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_groq import ChatGroq
from system_prompt import SYSTEM_PROMPT

load_dotenv()
MODEL = getenv("MODEL_NAME")
EMBED_MODEL = getenv("EMBED_MODEL")
GROQ_API_KEY = getenv("GROQ_API_KEY")
DOCUMENT_PATH = os.path.join(os.path.dirname(__file__), "mcp.pdf")

if not GROQ_API_KEY:
    raise ValueError("GROQ_API_KEY is not set in the environment variables.")

llm = ChatGroq(model=MODEL, temperature=0)
collection = chromadb.Client().create_collection(name="knowledge_base")
all_chunks: list[str] = []

def get_embedding(text):
    return ollama.embeddings(model=EMBED_MODEL, prompt=text)["embedding"]

def _chunk_to_text(content):
    if isinstance(content, str):
        return content
    if isinstance(content, list):
        parts = []
        for item in content:
            if isinstance(item, dict):
                parts.append(item.get("text", ""))
            else:
                parts.append(str(item))
        return "".join(parts)
    return str(content)

def _prepare_retrieval_query(user_input):
    for line in user_input.splitlines():
        line = line.strip()
        if line:
            return line
    return user_input.strip()

def _keyword_tokens(text):
    stop = {"who","what","when","where","why","how","is","are","do",
            "does","did","can","could","would","should"}
    return [token for token in re.findall(r"[a-z0-9]+", text.lower())
            if token not in stop]

def build_vector_store():
    print(f"Indexing: {DOCUMENT_PATH}...")
    docs = PyPDFLoader(DOCUMENT_PATH).load()
    chunks = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200
    ).split_documents(docs)
```

```
    for i, chunk in enumerate(chunks):
        all_chunks.append(chunk.page_content)
        collection.add(
            ids=[f"chunk_{i}"],
            embeddings=[get_embedding(chunk.page_content)],
            documents=[chunk.page_content]
        )
    print(f"Indexed {len(chunks)} chunks.")

def retrieve_context(question, top_k=15):
    retrieval_query = _prepare_retrieval_query(question)
    results = collection.query(
        query_embeddings=[get_embedding(retrieval_query)], n_results=top_k
    )
    semantic = (results["documents"] or [[]])[0]
    semantic_set = set(semantic)
    keywords = _keyword_tokens(retrieval_query)
    scored = sorted(all_chunks,
                    key=lambda c: sum(1 for kw in keywords if kw in c.lower()),
                    reverse=True)
    keyword_hit = [c for c in scored[:10]
                   if c not in semantic_set and any(kw in c.lower() for kw in keywords)]
    return "\n\n".join(semantic + keyword_hit)

def get_streaming_response(user_input):
    context = retrieve_context(user_input)
    messages = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=f"Context:\n{context}\n\nQuestion: {user_input}")
    ]
    yield ("thinking", f"Retrieved relevant document chunk. Asking {MODEL}...\n")
    for chunk in llm.stream(messages):
        text = _chunk_to_text(chunk.content)
        if text:
            yield ("response", text)

build_vector_store()

if __name__ == "__main__":
    print(f"RAG Chatbot using {MODEL} is ready. Ask your questions!")
    while True:
        user_input = input("User: ")
        if user_input.lower() == "exit":
            break
        for kind, text in get_streaming_response(user_input):
            print(text, end="", flush=True)
        print("\n")
```

3. Code Walkthrough for Beginners

3.1 Imports and configuration

The first block loads all the third-party libraries the chatbot depends on and reads configuration from a `.env` file.

```
from dotenv import load_dotenv
from os import getenv
import ollama, chromadb, re, os
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_groq import ChatGroq

load_dotenv()                        # reads .env into os.environ
MODEL       = getenv("MODEL_NAME")   # e.g. "llama3-8b-8192"
EMBED_MODEL = getenv("EMBED_MODEL")  # e.g. "nomic-embed-text"
GROQ_API_KEY = getenv("GROQ_API_KEY")
```

What is a .env file? It is a plain text file that stores secret values like API keys outside of your source code. `python-dotenv` reads it automatically when you call `load_dotenv()`.

- `ollama` — runs open-source LLMs locally on your machine, here used only for creating text embeddings.
- `chromadb` — an in-memory vector database that stores embeddings and finds the most similar ones on demand.
- `langchain_groq` — a wrapper around Groq's API, which runs large language models very fast on specialised chips.

3.2 Global objects

```
llm = ChatGroq(model=MODEL, temperature=0)

collection = chromadb.Client().create_collection(name="knowledge_base")
all_chunks: list[str] = []
```

Three objects are created once and reused throughout the program:

- `llm` — the language model. `temperature=0` makes answers deterministic (no randomness).
- `collection` — an in-memory ChromaDB collection. Think of it as a special list that can search by meaning, not just by text.
- `all_chunks` — a plain Python list that stores the raw text of every chunk for keyword searching.

3.3 Embedding helper

```
def get_embedding(text):
    return ollama.embeddings(model=EMBED_MODEL, prompt=text)["embedding"]
```

An **embedding** is a list of hundreds of numbers that encodes the meaning of a sentence. Two sentences with similar meanings will have similar embeddings (their number-lists will be close together in high-dimensional space). This function asks the local Ollama model to compute that list for any text.

3.4 build_vector_store — the indexing phase

```
def build_vector_store():
    docs = PyPDFLoader(DOCUMENT_PATH).load()
    chunks = RecursiveCharacterTextSplitter(
```

```
        chunk_size=1000, chunk_overlap=200
    ).split_documents(docs)
    for i, chunk in enumerate(chunks):
        all_chunks.append(chunk.page_content)
        collection.add(
            ids=[f"chunk_{i}"],
            embeddings=[get_embedding(chunk.page_content)],
            documents=[chunk.page_content]
        )
```

This runs once at startup and has three steps:

- **Load** — PyPDFLoader reads every page of the PDF and returns a list of LangChain Document objects.
- **Split** — RecursiveCharacterTextSplitter breaks each page into 1000-character chunks that overlap by 200 characters (so no sentence is cut off mid-thought). See Section 4 for details.
- **Index** — for each chunk, Ollama computes an embedding and ChromaDB stores both the embedding and the original text. The plain text is also appended to all_chunks for keyword search.

3.5 retrieve_context — hybrid search

```
def retrieve_context(question, top_k=15):
    # 1. Semantic search
    results = collection.query(
        query_embeddings=[get_embedding(question)], n_results=top_k
    )
    semantic = (results["documents"] or [[]])[0]
    semantic_set = set(semantic)

    # 2. Keyword search
    keywords = _keyword_tokens(question)   # removes stop words
    scored = sorted(all_chunks,
        key=lambda c: sum(1 for kw in keywords if kw in c.lower()),
        reverse=True)
    keyword_hit = [c for c in scored[:10]
        if c not in semantic_set
        and any(kw in c.lower() for kw in keywords)]

    return "\n\n".join(semantic + keyword_hit)
```

This is the heart of the RAG system. It runs two searches and combines them:

- **Semantic search** — embeds the question and asks ChromaDB for the top 15 chunks whose embeddings are mathematically closest. This finds passages that mean the same thing even if the exact words differ.
- **Keyword search** — removes common words ("what", "is", "how" …) from the question, then scores every chunk by how many of the remaining keywords it contains. This catches exact technical terms like "MCP" that embeddings sometimes under-rank.
- **Deduplication** — keyword results already in the semantic results are dropped via `semantic_set`, so the LLM never sees the same text twice.

3.6 get_streaming_response — generation

```
def get_streaming_response(user_input):
    context = retrieve_context(user_input)
    messages = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=f"Context:\n{context}\n\nQuestion: {user_input}")
    ]
    yield ("thinking", f"Retrieved relevant document chunk...")
    for chunk in llm.stream(messages):
```

```
        text = _chunk_to_text(chunk.content)
        if text:
            yield ("response", text)
```

This function is a **generator** — it uses `yield` instead of `return`, which means it produces values one at a time without waiting for the full response. This is what makes the output stream word-by-word rather than arriving all at once.

- `SystemMessage` — sets the LLM's persona and rules (the contents of system_prompt.py). The model always follows these instructions.
- `HumanMessage` — the user's question, preceded by all the retrieved context. The LLM's job is to answer the question using only that context.
- `llm.stream()` — instead of waiting for the full answer, ChatGroq sends tokens as soon as they are generated. Each `chunk` in the loop is a small piece of the final response.

4. How RecursiveCharacterTextSplitter Works

The name sounds intimidating, but the idea is simple: try to split at natural language boundaries first. Only if a piece is still too large does it fall back to a finer separator.

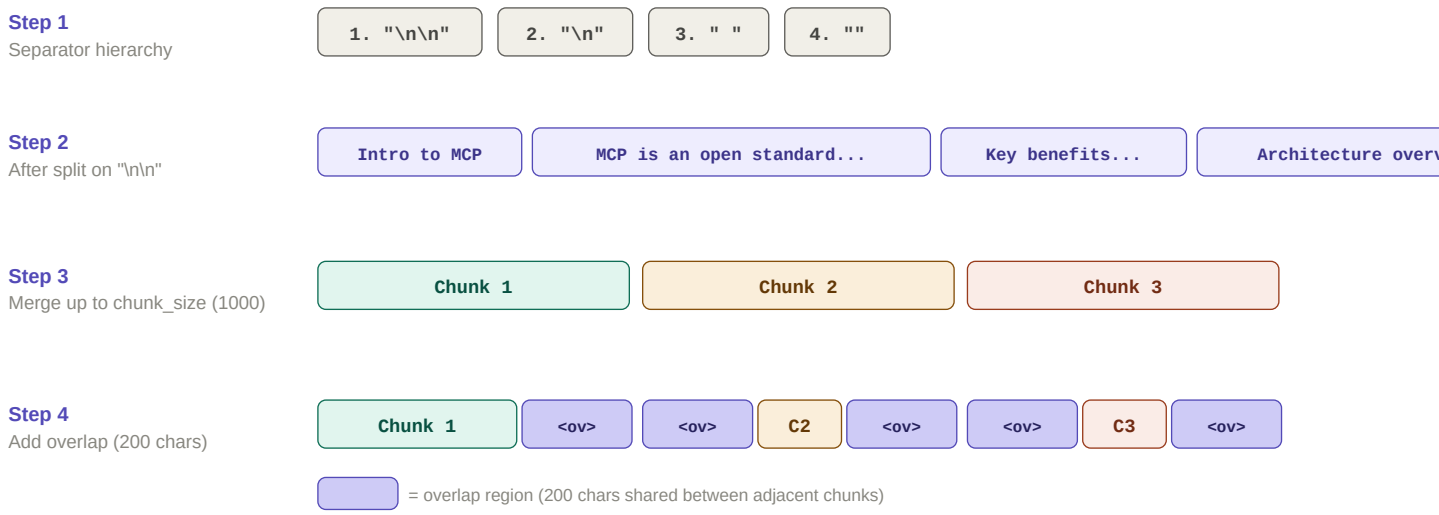

Figure 2 — The four stages of splitting. Purple = raw paragraphs after the first split; teal/amber/coral = final merged chunks; violet dashes = 200-char overlap regions.

4.1 The separator hierarchy

The splitter is given a priority-ordered list of separators. It always tries the coarsest one first:

```
# Default separators, tried in this order:
["\\n\\n",   # paragraph break — keeps whole paragraphs together
 "\\n",     # line break — splits within a paragraph
 " ",      # space — splits at word boundaries
 ""]       # character — last resort, one char at a time
```

Most PDF pages are short enough that the very first separator (`\n\n`) produces paragraphs that fit inside 1000 characters. The recursion to finer separators only happens for unusually long paragraphs (common in academic or legal documents).

4.2 Greedy merging

After splitting, the pieces are merged back together greedily: the splitter keeps adding pieces to the current chunk until the next piece would push it over `chunk_size=1000`, then it starts a new chunk. This avoids tiny chunks that give the LLM insufficient context.

4.3 The overlap

The last `chunk_overlap=200` characters of each chunk are copied into the beginning of the next chunk. This is the safety net: a sentence that sits exactly on a chunk boundary will appear in full in at least one of the two chunks. Without overlap, the retrieval system might return a chunk that starts mid-sentence and confuses the LLM.

4.4 Why this matters for retrieval

Chunk size is one of the most important hyperparameters in a RAG system. Too large and each chunk contains too many topics, diluting the embedding. Too small and individual chunks lose context. The 1000/200 defaults used here are a reasonable starting point for technical documentation like MCP.

- If answers seem too vague, try reducing chunk_size to 500 — the LLM will receive more focused passages.
- If answers seem incomplete or cut off mid-explanation, try increasing chunk_size to 1500.
- If you see repeated text in the LLM context, reduce chunk_overlap.